DASC7606 -8
Deep Learning

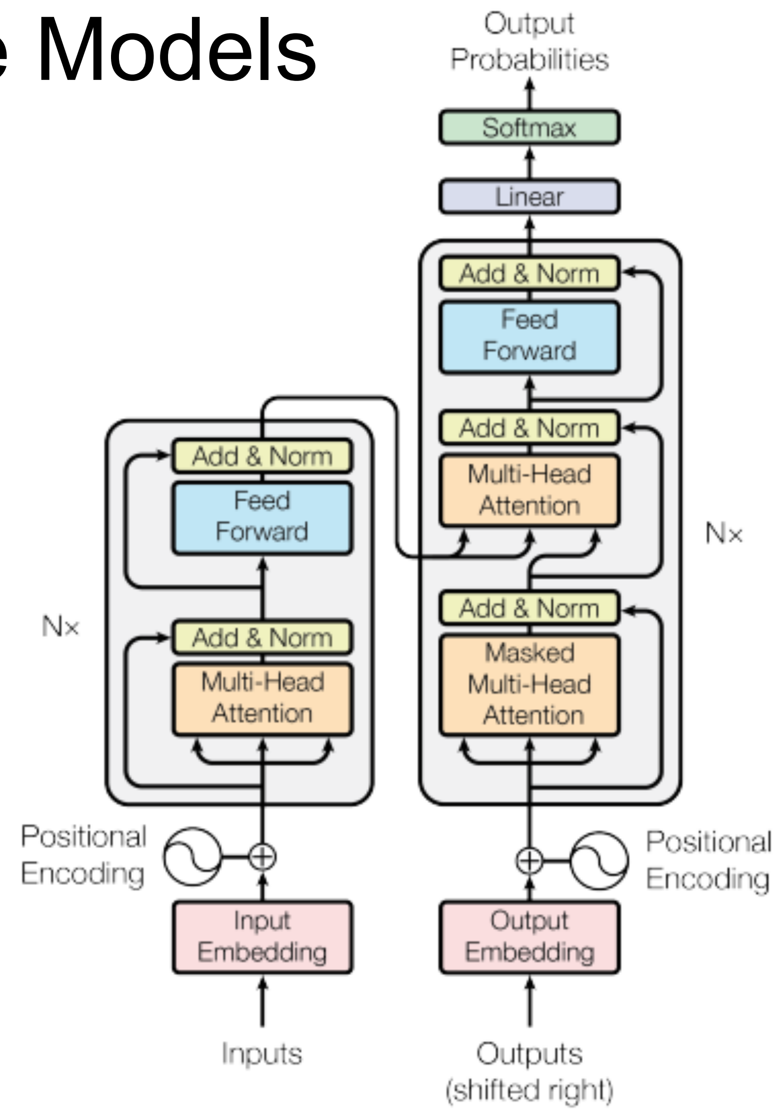
Large Language Models (LLM)
and
"Generative Pre-trained Transformer" GPT

Dr. Bethany Chan
Professor Francis Chin

2025

Large Language Models

- Word Embedding
  - Word2Vec (CBOW)
  - Evaluation
  - ELMo
- Seq2Seq – Machine Translation
- Attention and Transformer
- BERT
  - Architecture and pre-training
  - Using BERT for word embeddings
  - Using BERT for other NLP tasks
  - Tokenization
- Language Model and LLM
- Open AI, GPTs, ChatGPT
- BART - another Pre-trained Large Language Model (LLM)
- Prompting and In-Context Learning



BERT (**B**idirectional **E**ncoder **R**epresentations from **T**ransformers)

Both ELMo and BERT

are cartoons from Sesame Street

ELMo = **E**mbeddings from **L**anguage **Mo**del

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, 2018

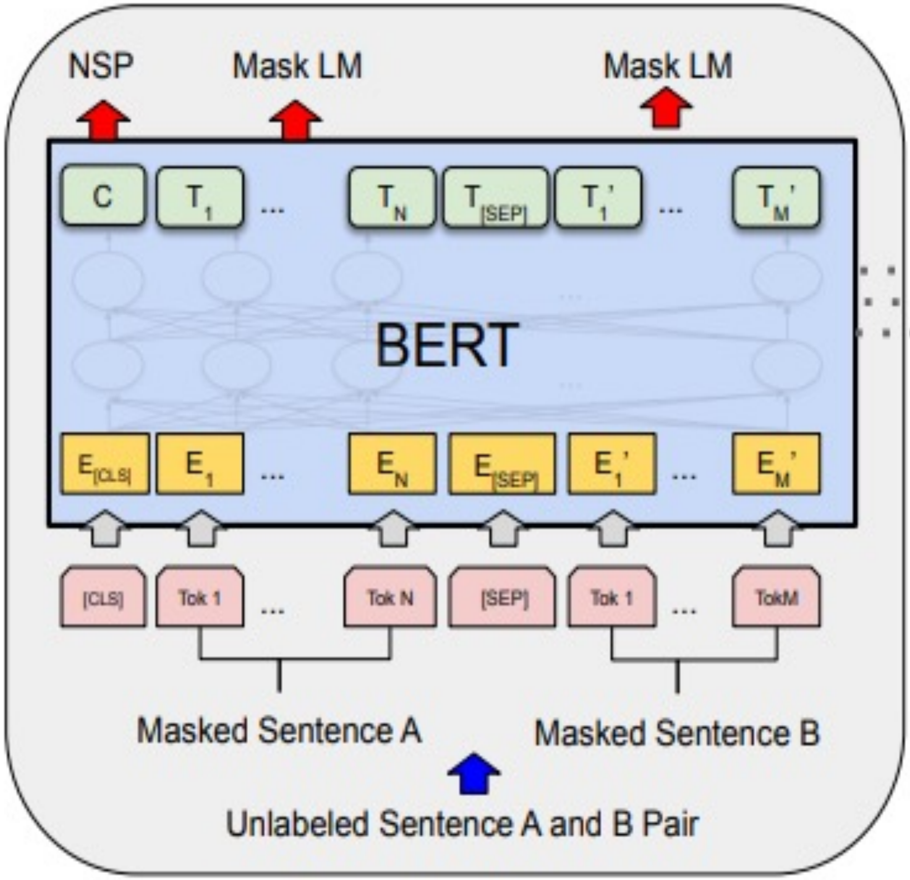
BERT: Architecture and pre-training

BERT uses the encoder mechanism of the Transformer.

Pre-training of BERT

Masked LM Training: Mask 15% of the words, predict the masked words, based on the non-masked words in the sequence

Next Sentence Prediction (NSP): Binary classifier is trained to predict whether one sentence is the next sentence of another (using 50% subsequent pairs and 50% random).

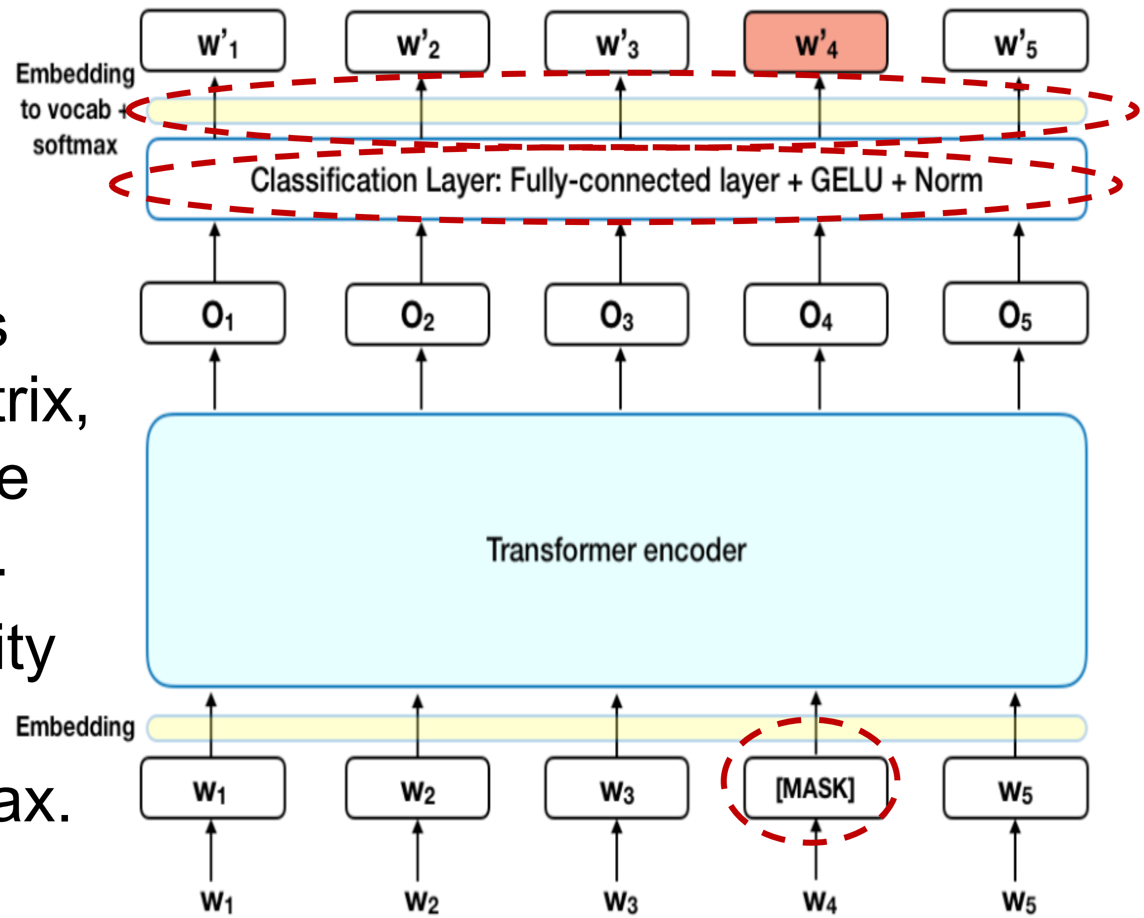


Pre-training

BERT – Mask Training

Mask Training: Mask 15% of the words, predict the masked words, based on the non-masked words in the sequence (CBOW)

- Add a classification layer on top of the encoder output.
- Multiply output vectors by the embedding matrix, transform them into the vocabulary dimension.
- Calculate the probability of each word in the vocabulary with softmax.



BERT: Architecture and pre-training

Advantages

- Masked token prediction
  better than next token prediction
  (e.g., RNN which is only sequentially and not the whole context)
- Transformer attention gives varying weight to different parts of input data
  - ‘reading’ often associates with ‘hate’
  - Attention makes better prediction
    e.g., if ‘Jacob’ is an avid reader,
    more attention to ‘Jacob’
    than to ‘reading’ and
    choose ‘love’ instead of ‘hate’.

Masked-language-modeling

The model is given a sequence of words with the goal of predicting a ‘masked’ word in the middle.

Example
Jacob [mask] reading

Jacob *fears* reading
Jacob *loves* reading
Jacob *enjoys* reading
Jacob *hates* reading

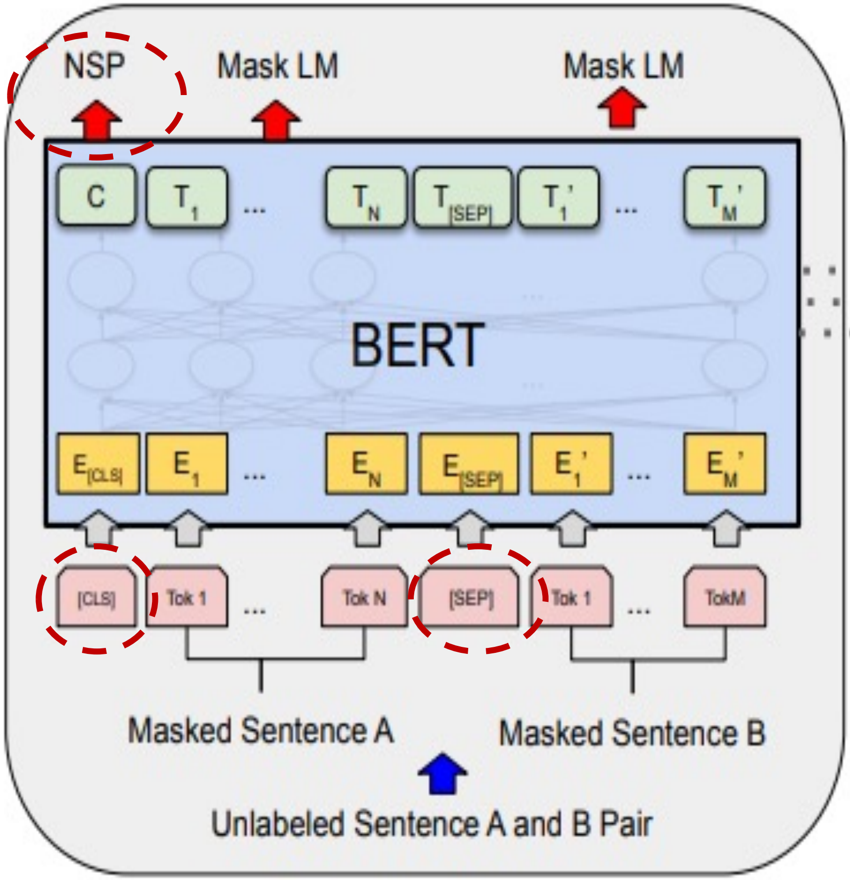
Masking word prediction

Learn from reconstructing the input

- **Fact**: University of Hong Kong is located in _______ , Hong Kong.
- **Syntax**: I put ___ fork down on the table.
- **Coreference**: The woman walked across the street, checking for traffic over ____ shoulder.
- **Semantics/topics**: I went to the ocean to see the fish, turtles, seals and ________.
- **Sentiment**: Overall, the food was delicious and the service was great, the restaurant was ______.
- **Reasoning**: Grace went to the kitchen to make tea, Beth was standing next to Grace. Beth later left the ______.
- **Arithmetic**: The answer of 3 multiply 9 is _______.

BERT Training: Next Sentence Prediction

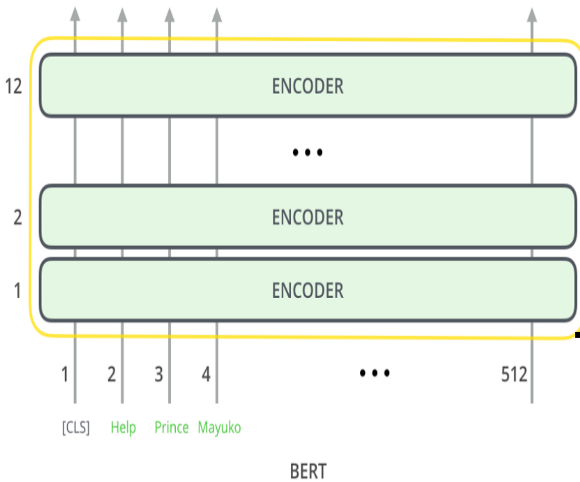
- Input pairs of sentences ([CLS] token inserted at the beginning, [SEP] token at the end of each sentence)
- Learn if the second sentence follows the first sentence
- 50% are subsequent pairs, the other 50% are random
- Use a simple classification layer to calculate probability IsNextSequence with Softmax.
- NSP helps for tasks based on relationship between two sentences.

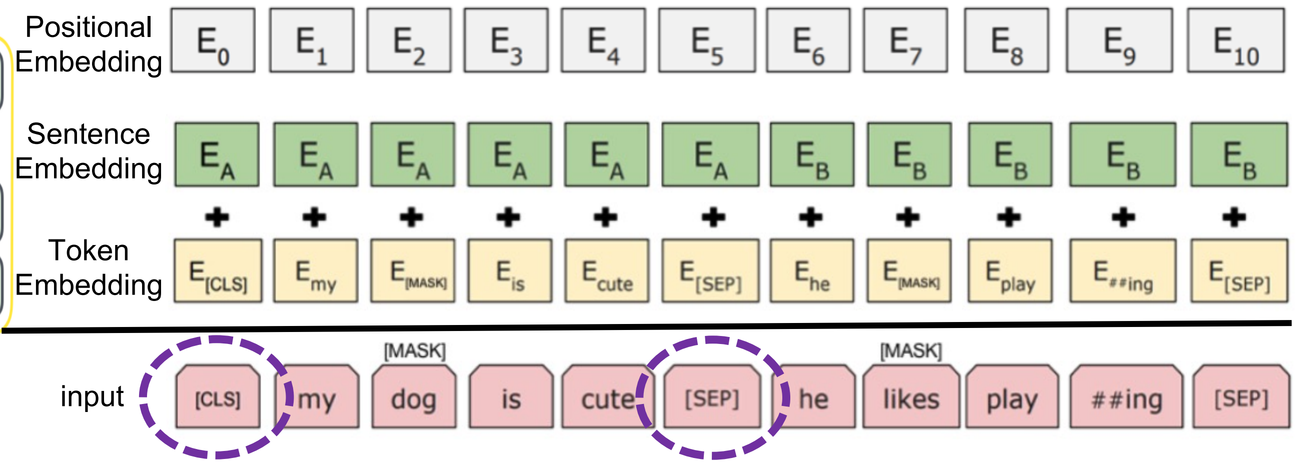


Pre-training

RoBERTa (A Robustly Optimized BERT Approach) more data by dynamic masking, no Next Sentence Prediction.

BERT: Architecture and pre-training





BERT-Base: 12 encoder blocks, hidden layer size = 768

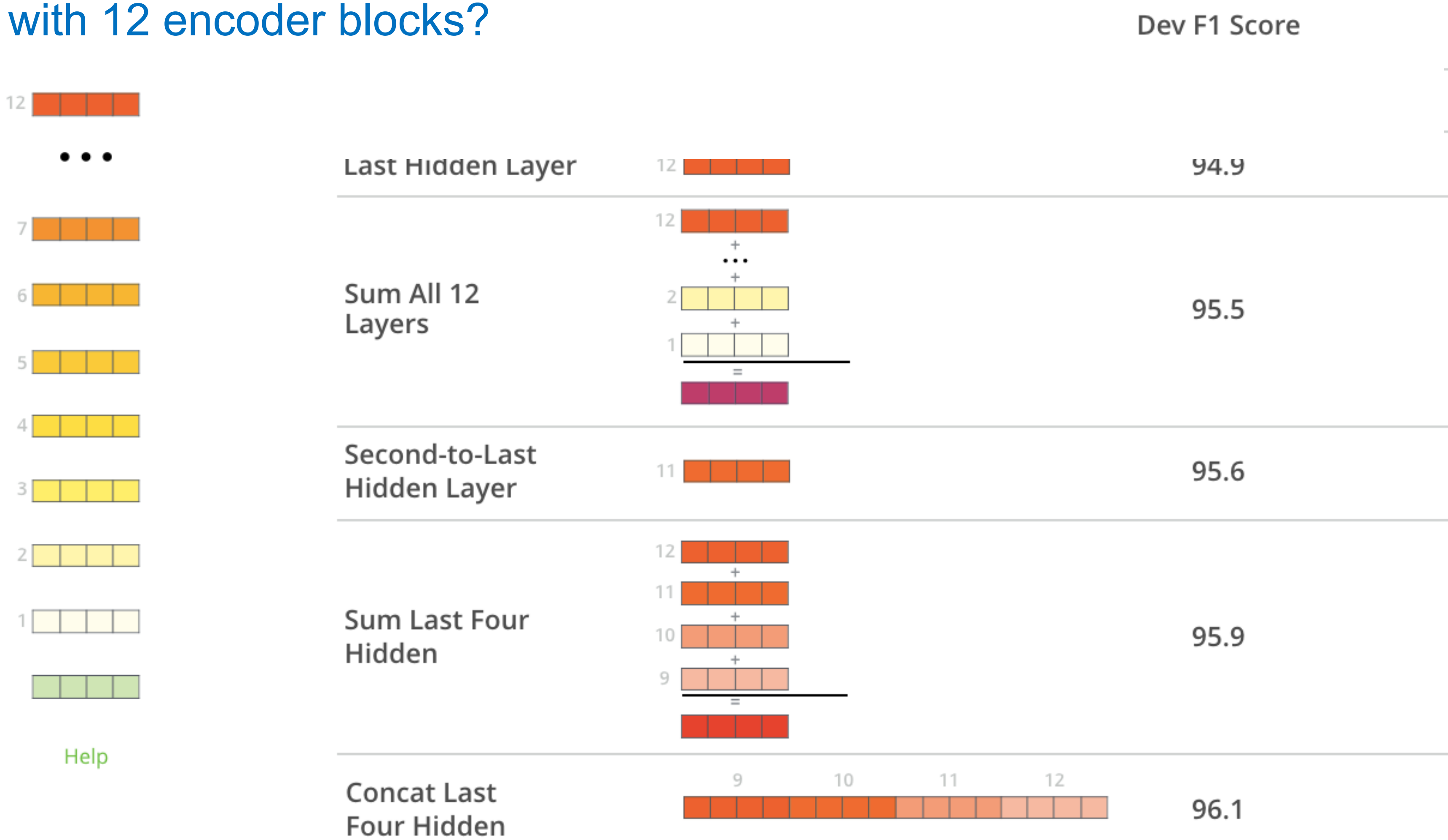
Input format

- **Input layer**: [CLS] – first token; [SEP] – delimited token
- **Token embeddings:** vocabulary IDs for each of the tokens.
- **Sentence Embeddings:** series of 0s/1s to distinguish sentences
- **Positional embeddings:** each word position in the sequence.

Each output per token from each layer can be used as a **word embedding**, e.g., the last 4 layers were used. (vector size: concatenation 4x768=3072 or summing 768)

Using BERT for Word Embeddings

What is the best contextualized embedding with 12 encoder blocks?



Word Embedding: the last 4 layers give the best result.
(vector size: concatenation 4x768=3072 or summing 768).

Using BERT for Word Embeddings: Contextually Dependent Vectors

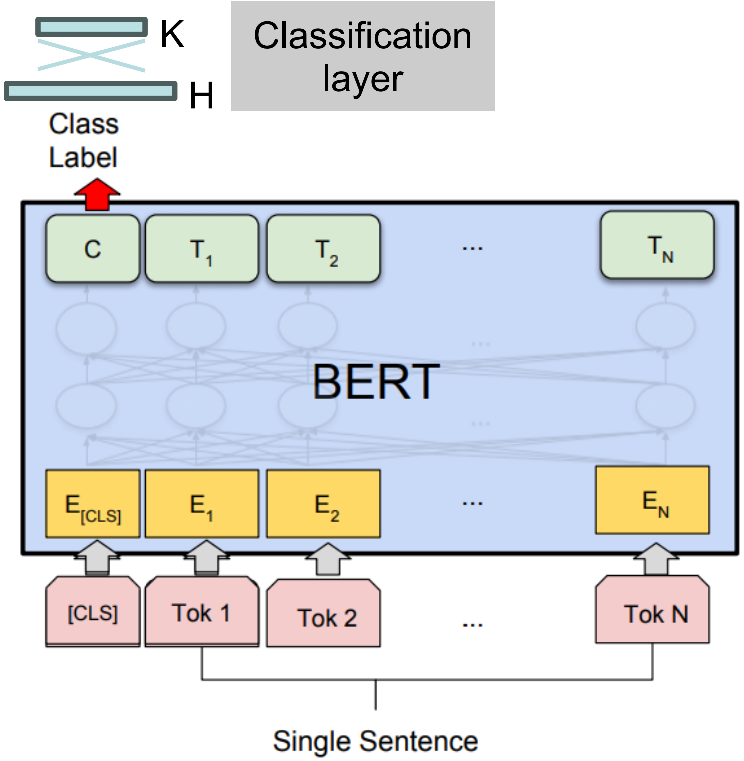
- [CLS]^{0} After^{1} stealing^{2} money^{3} from^{4} the^{5} **bank**^{6} **vault**^{7},^{8} the^{9} **bank**^{10} **robber**^{11} was^{12} seen^{13} fishing^{14} on^{15} the^{16} Mississippi^{17} **river**^{18} **bank**^{19}.^{20} [SEP]^{21}
- Instances of “bank” at positions 6, 10, and 19, whose word vectors are summing the last 4 hidden layers
- First 5 vector values of their 768-vectors for each instance of "bank”
  - **bank**^{6} vault ([3.3596, -2.9805, -1.5421, 0.7065, 2.0031, …])
  - **bank**^{10} robber ([2.7359, -2.5577, -1.3094, 0.6797, 1.6633, …])
  - river **bank**$^{19.}$ ([1.5266, -0.8895, -0.5152, -0.9298, 2.8334, …])

Observations:

- Word vectors for each instance of “bank” are different
- Cosine Vector Similarity of
  - **bank**^{6} vault and **bank**^{10} robber = 0.94
  - **bank**^{10} robber and **river** bank^{19} = 0.69

Using BERT for other NLP tasks (Fine tune)

- BERT can be used for a wide variety of language tasks, while only adding a small layer to the core model.
- In the fine-tuning training, most of the hyper-parameters stay the same as in BERT training.



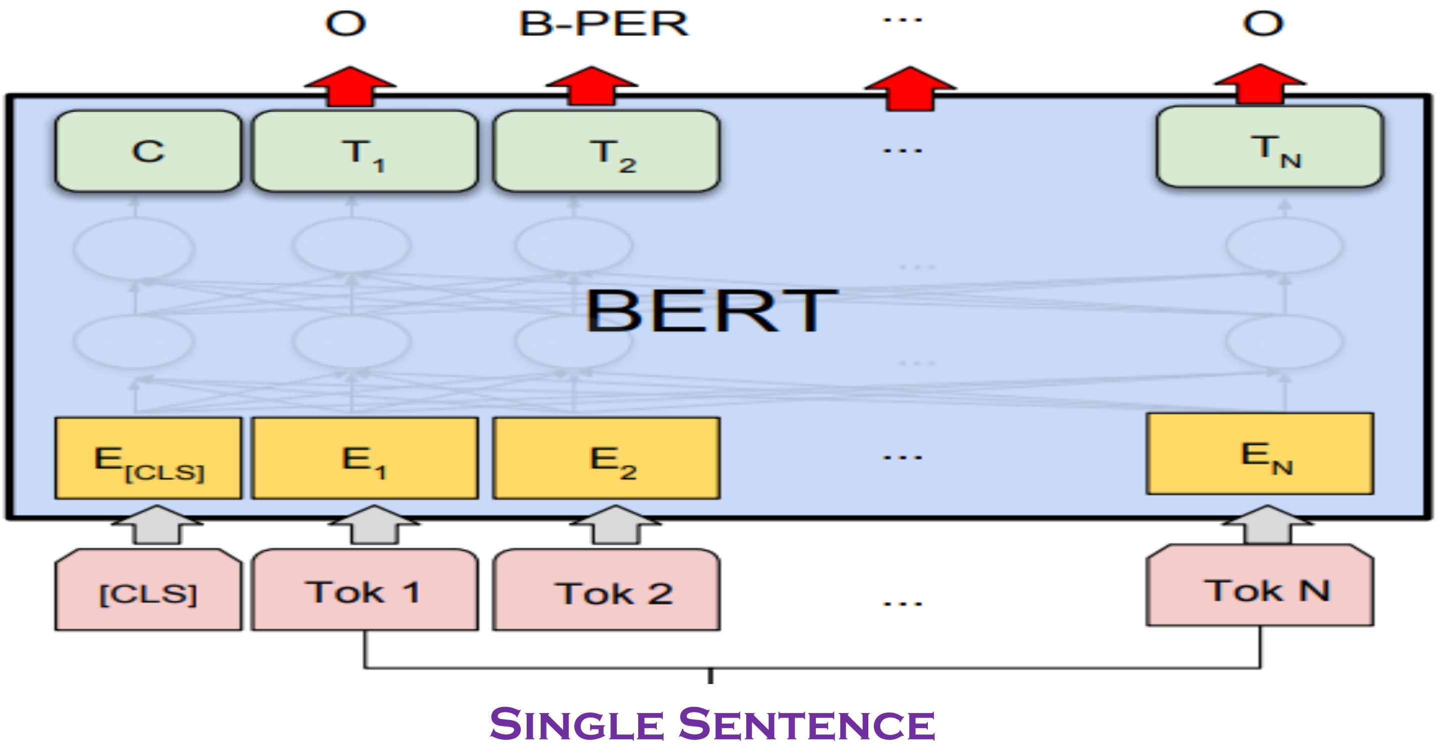
Classification tasks

(single sentence sentiment analysis inferencing, semantic similarity, …)

- Input sentence starts with **[CLS]**, separate/end with **[SEP]**
- A classification layer is added on the hidden states of **[CLS]** of dimension of **K x H**, where
  K = number of classifier labels,
  H = size of the hidden state.
- Label probabilities are computed with a standard softmax.

Single Sentence Tagging

Named Entity Recognition (NER) marks entities (Person, Organization, Date, etc.) in the text



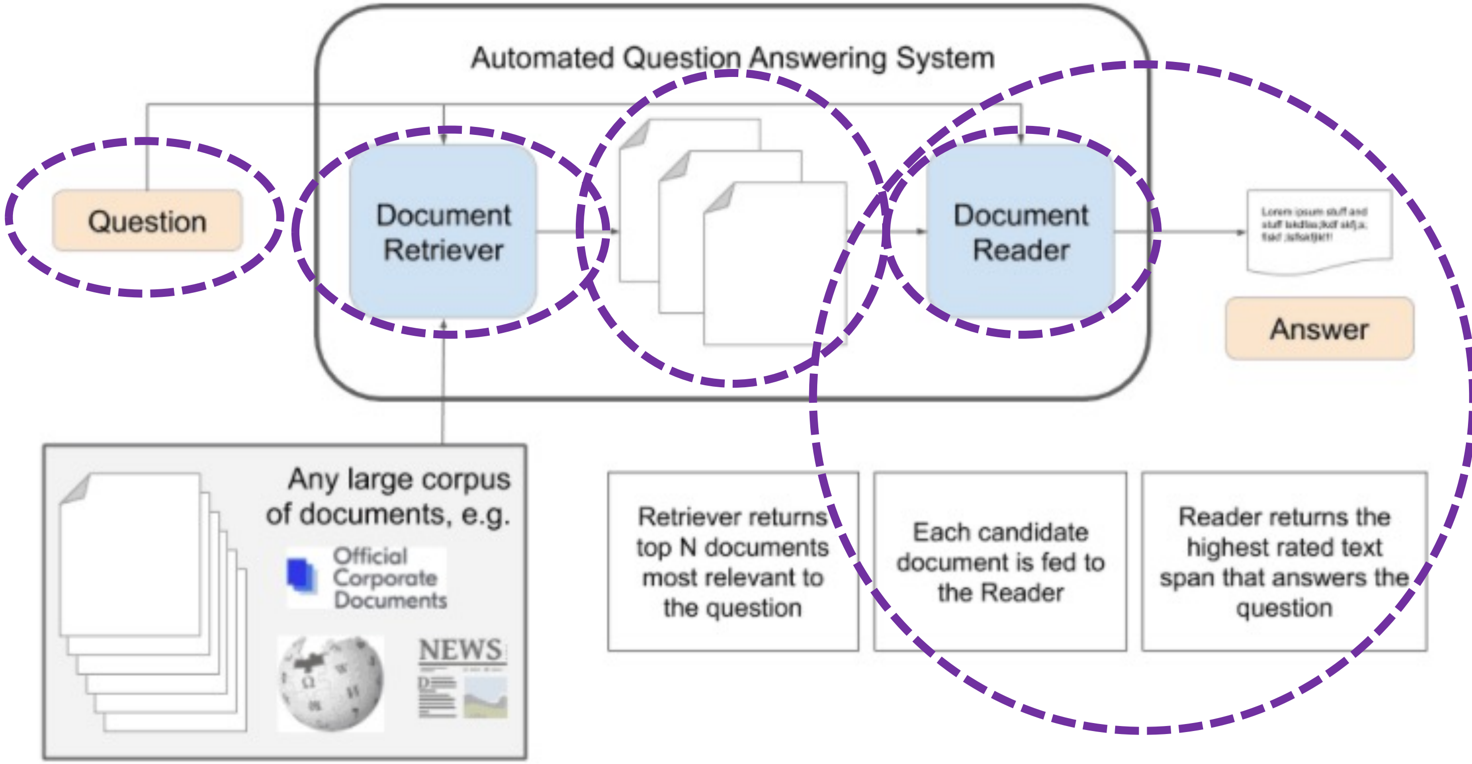
- A tag is predicted for each word
- Final hidden states (transformer output) of every input token is fed to the classification layer to get a prediction for every token.

Question and Answering System

Task: retrieve the answer to a question from a given text

Extractive Approach consist of retrieval and a reader

1. Query processing – extract keywords by part-of-speech
2. Document retrieval – search documents, retrieval contents
3. Passage retrieval – select relevant passages (smaller units)
4. Answer extraction - extract answer from passages (BERT)



Question and Answering System

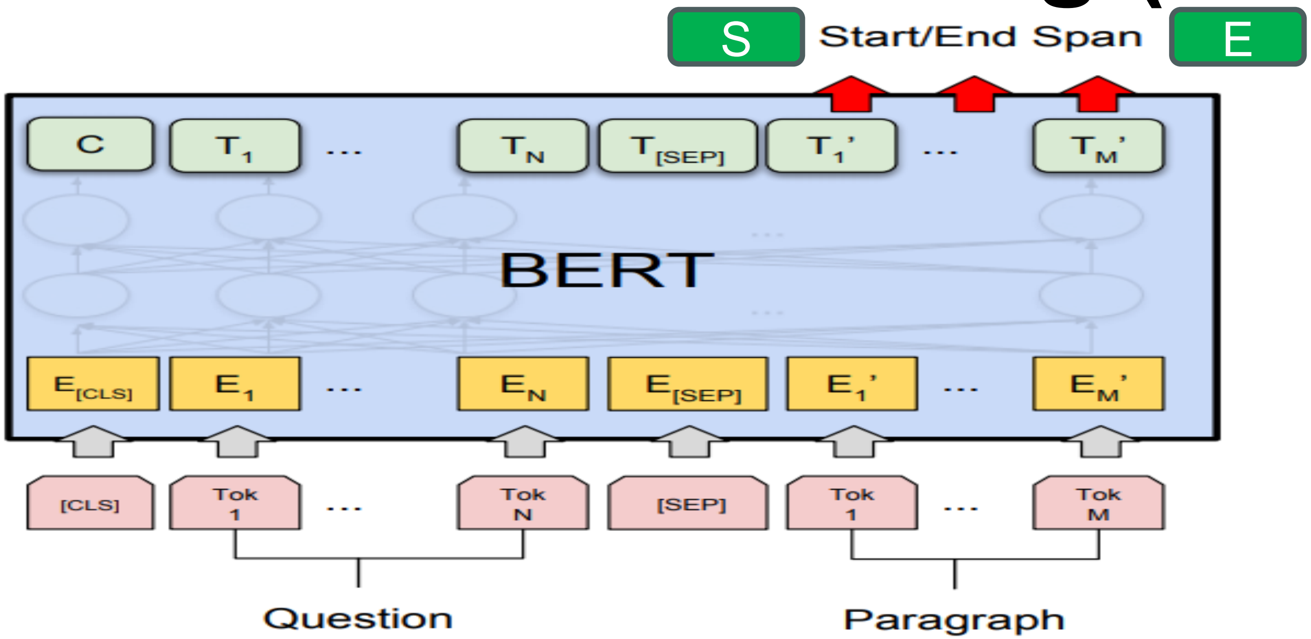
Q: Of what group in the periodic table is oxygen a member?

Q: What is the atomic number of oxygen?

Q: Is oxygen abundant?

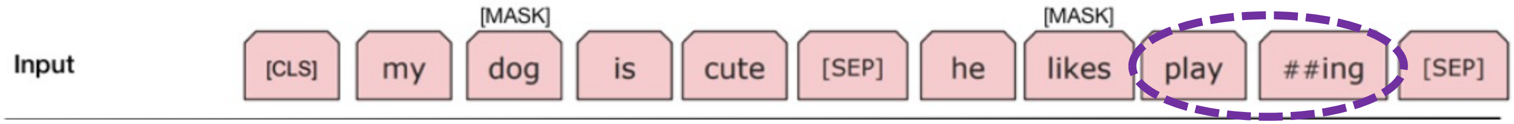
Oxygen is a chemical element with symbol O and atomic number 8. It is a member of the chalcogen group on the periodic table and is a highly reactive nonmetal and oxidizing agent that readily forms compounds (notably oxides) with most elements. By mass, oxygen is the third-most abundant element in the universe, after hydrogen and helium. At standard temperature and pressure, two atoms of the element bind to form dioxygen, a colorless and odorless diatomic gas with the formula O
2. Diatomic oxygen gas constitutes 20.8% of the Earth's atmosphere. However, monitoring of atmospheric oxygen levels show a global downward trend, because of fossil-fuel burning. Oxygen is the most abundant element by mass in the Earth's crust as part of oxide compounds such as silicon dioxide, making up almost half of the crust's mass.

Question and Answering (Reader)



- Input: question and context paragraph (sentence pair task)
- Output: starting token and end token from the paragraph
  that answer the question (Token **i** with output $\mathbf{T}_i$)
- Learn two new parameters:
  start vector S and **end vector E** (same size as Token $\mathbf{T}_i$)
- Prob (Token **i** is answer start) = softmax ($\mathbf{S} \cdot \mathbf{T}_i$)
  Prob (Token **i** is answer end) = softmax ($\mathbf{E} \cdot \mathbf{T}_i$)

Tokenization



Tokenization is the process of splitting a word, phrase, sentence, paragraph, or text documents into smaller units. (sentence, word, character tokenization)

Subword Tokenization (a balance between word-based and character-based tokenization) breaks words into smaller and meaningful pieces based on frequency and structure (Byte-Pair or WordPiece Tokenization)

Example: “embeddings” → ['em', '##bed', '##ding', '##s’]

- ## denotes a preceding subword, e.g., ‘##bed’ token denotes subword with suffix ‘bed’, which is also a standalone token.

Tokenization in BERT

BERT tokenizer model (vocabulary size 30,000) contains :

1. Whole words
2. Subwords occurring at the front of a word or in isolation ("em" in "embeddings" or standalone word "em" – letter "m")
3. Subwords preceded by '##' (not at the front of a word)
4. Individual characters

Advantage: Reduce vocabulary size (size of the model) and possible to process unseen words by its subwords.

To tokenize a word

– first checks if the whole word is in the vocabulary.

– If not, break the word into the largest possible subwords in the vocabulary,

– As last resort, decompose the word into individual characters

(Note that a word can always be represented as a collection of characters)

Large Language Models

- Word Embedding
  - Word2Vec (CBOW)
  - Evaluation
  - ELMo
- Seq2Seq – Machine Translation
- Attention and Transformer
- BERT
  - Architecture and pre-training
  - Using BERT for word embeddings
  - Using BERT for other NLP tasks
  - Tokenization
- Language Model and LLM
- Open AI, GPTs, ChatGPT
- BART - anther Pre-trained Large Language Model (LLM)
- Prompting and In-Context Learning

Natural Language Processing (NLP)

- 1970-80's, early NLP is rule-based (by hand)
- 1990 - Using statistical inference to learn rules about "validity" through large collections of documents
- A ***language model*** (LM) is a probabilistic model that predicts the order of words in a sentence
  - P("I love you") is high; P("eat pen north") is low
- LM deduces "rules" about **validity** through statistics
  - Spelling - "breakfast", "audio", "chulling" P(w) =0
  - Grammar - "I go", "he goes", "one book", "two books", "one water" P("one water") =0
  - Structure - (subject – verb – object,…)

Statistical LM instead of Rule-based LM

Given a word string, $S = w_1 w_2 \ldots w_n$, LM computes

Probability (next word)

- $P(S) = P(w_1 w_2 \ldots w_n) = P(w_1)\, P(w_2|w_1) \ldots P(w_n|w_1 \ldots w_{n-1})$

Language Model for Style

All work of Shakespeare → Language model →

KING LEAR:
O, if you were a feeble sight, the courtesy of your law,
Your sight and several breath, will wear the gods
With his heads, and my hands are wonder'd at the deeds,
So drop upon your lordship's head, and your opinion
Shall be against your honour.

Base on LM (statistics), the next word is generated according to probability $P(w_n|w_1 \ldots w_{n-1})$

Language Model for Style

Computer Science textbook and Bible → Language model →

37:29 The righteous shall inherit the land, and leave it for an inheritance unto the children of Gad according to the number of steps that is linear in *b*.

hath it not been for the singular taste of old Unix, "new Unix" would not exist.

Language Model Development

2014	2017	2018	2018-2020	2022	2022	2023	2025
Seq2Seq	Transformer	Elmo, BERT	GPT-1,2,3	GPT3.5	ChatGPT	GPT4	GPT5

- GPT stands for "Generative Pre-trained Transformer"
- Generative through Language Model $P(s_j|s_1 \ldots s_{j-1})$, decoder of Transformer
- Prior to this work, NLP models (e.g., BERT combines Transformer with pretraining and self-supervised learning) were trained specifically on a particular task, like sentiment classification, textual entailment etc., using supervised learning.

  Limitations:

  – large amount of annotated data for learning a particular task which is often not easily available

  – fail to generalize for tasks other than what they have been trained for.

Large Language Models

- Word Embedding
  - Word2Vec (CBOW)
  - Evaluation
  - ELMo
- Seq2Seq – Machine Translation
- Attention and Transformer
- BERT
  - Architecture and pre-training
  - Using BERT for word embeddings
  - Using BERT for other NLP tasks
  - Tokenization
- Language Model and LLM
- Open AI, GPTs, ChatGPT
- BART - anther Pre-trained Large Language Model (LLM)
- Prompting and In-Context Learning

Open AI:
GPT

Generative Pre-trained Transformer

Reference materials:
Aligning language models to follow instructions,
https://openai.com/research/instruction-following, Jan 2022
https://chat.openai.com/ OpenAI. Nov 2022.
https://openai.com/blog/introducing-text-and-code-embeddings
https://medium.com/walmartglobaltech/the-journey-of-open-ai-gpt-models-32d95b7b7fb2
HKU Talk by Qi Liu, “How ChatGPT Obtains Its Ability and Its Impacts”, March 2023
GPT-4 Technical Report, March 2023, https://arxiv.org/pdf/2303.08774.pdf

What is Open AI?

Artificial Intelligence research laboratory and deployment company by Elon Musk in 2016.

Mission for OpenAI:

- advance artificial general intelligence (AGI) to benefit humanity (*Elon Musk is very vocal about danger of AI*)
- non-profit to "capped" for-profit in 2019 (partnered with Microsoft after Elon Musk stepped down in 2018)
- E. Musk sued CEO Sam Altman in 2024 shifting focus from public benefit to profit maximization, Altman countersued Musk for "bad-faith tactics" to slow the company's progress
- Partner with Microsoft (copilot), Nvidia…

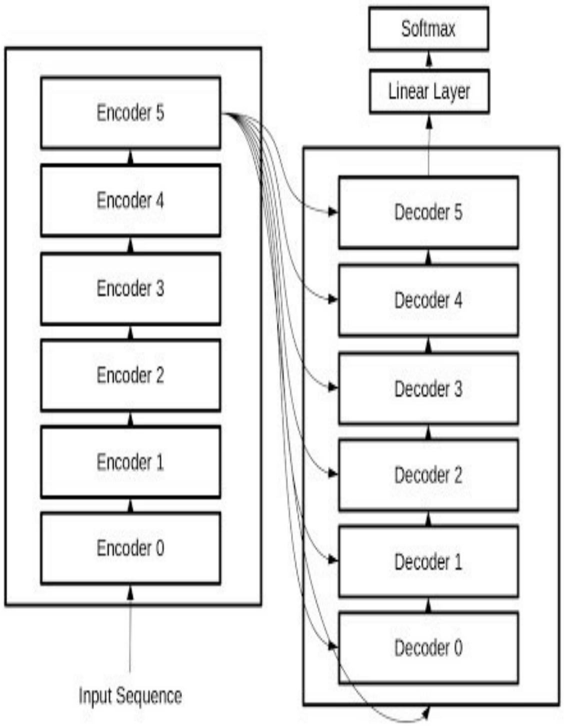
GPT performs a wide variety of natural language tasks (with about 30 engineers and researchers)

DALL.E creates realistic images and art from a description in natural language

SORA creates video from text

BERT vs GPT

- OpenAI's GPT uses only its decoder to generate outputs (uni-directional).
- BERT uses its encoder (bidirectional) to builds a language model.
- GPT is much larger than BERT
  - BERT-base – 110M; large – 340M (2018)
  - GPT-1 -2 -3 -4 – 110M, 1.5B, 175B, 100T?
- BERT and GPT-1 need data and fine-tune models for specific downstream tasks
- GPT-3's allow users to reprogram it through instructions (prompts) and few-shot learning.
- Few-shot learning – predictions of output is based on the input of a few examples for each new class.



“Generative Pre-trained Transformer”- GPT

https://medium.com/walmartglobaltech/the-journey-of-open-ai-gpt-models-32d95b7b7fb2

- GPT models compress world knowledge through language modeling (**decoder-only Transformer model**),
- GPT can recover (memorize) the semantics of world knowledge and serve as a general-purpose task solver.
- Two key points to the success are
  - **Data** - training language models to accurately predict next word
  - **Compute** - scaling up the size of language models
- GPT-1 first pretrain Transformer with unlabeled data through
  - Unsupervised Learning (text T, window size k, NN parameter θ)

$$L_1(T) = \sum_i log\, P(t_i | t_{i-k}, \ldots, t_{i-1}; \theta) \qquad (i)$$

- Then fine-tune model with examples on each downstream task, such as classification, sentiment…
  - Supervised fine-tuning (input $x_1,\ldots x_n$, observed label y)

$$L_2(C) = \sum_{x,y} log\, P(y | x_1, \ldots, x_n) \qquad (ii)$$

GPT-1 (2018)

- Used 12-layer decoder, 12 attention heads,
  117 million parameters and 768-dim word embedding
- Data from over 7,000 books (with mask)

For fine tuning on each task, inputs were transformed into ordered sequences (as BERT).

- Start and end tokens were added to the input sequences.
- A delimiter token added between different parts of example.
- For tasks like question answering, multiple choice questions... multiple sequences were sent for each task
  e.g. training example for question answering task comprised of sequences for context, question and answer.

Training: 100 epochs pretraining, 3 epochs for fine-tuning

GPT-2 (2019)

	Parameter	layer	dim	context token size	attention
• GPT-1	117M	12	768	512	12
• GPT-2	1.5B	48	1600	1024	20

>10 times more parameters, 10 times more data, i.e., 40GB text

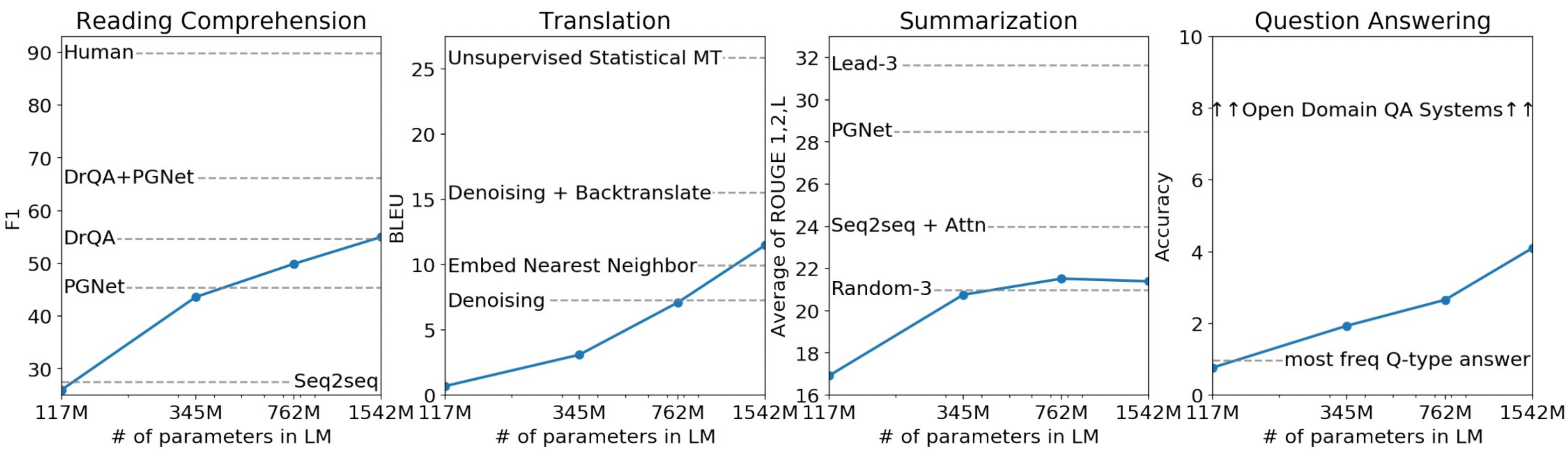
- One unsupervised model for multi-tasking.
  No fine tuning for specific tasks (no supervised learning)
- Language model - task conditioning: **P(output | input, task)** – different output for same input for different task tokens.
- The model tries to understand the nature of task and provide answers (to emulate **zero-shot task** transfer).
- Inferior performance compared with supervised fine-tuning state-of-the-art methods.
- **One-shot** example: English to Chinese translation task,
  input: I like to eat => 我喜欢吃 ;
  I want to go to market today =>

GPT2 : 我今天想去市场

GPT-2 Model Size and Perplexity

- Four language models were trained with 117M (same as GPT-1), 345M, 762M and 1.5B (GPT-2) parameters.
- Each subsequent model had better performance than previous one
- Performance (perplexity) improves with increase in no. of parameters (larger model) and longer training time on the same-dataset

  i.e., GPT-2 under fitted the dataset.



Training LLM

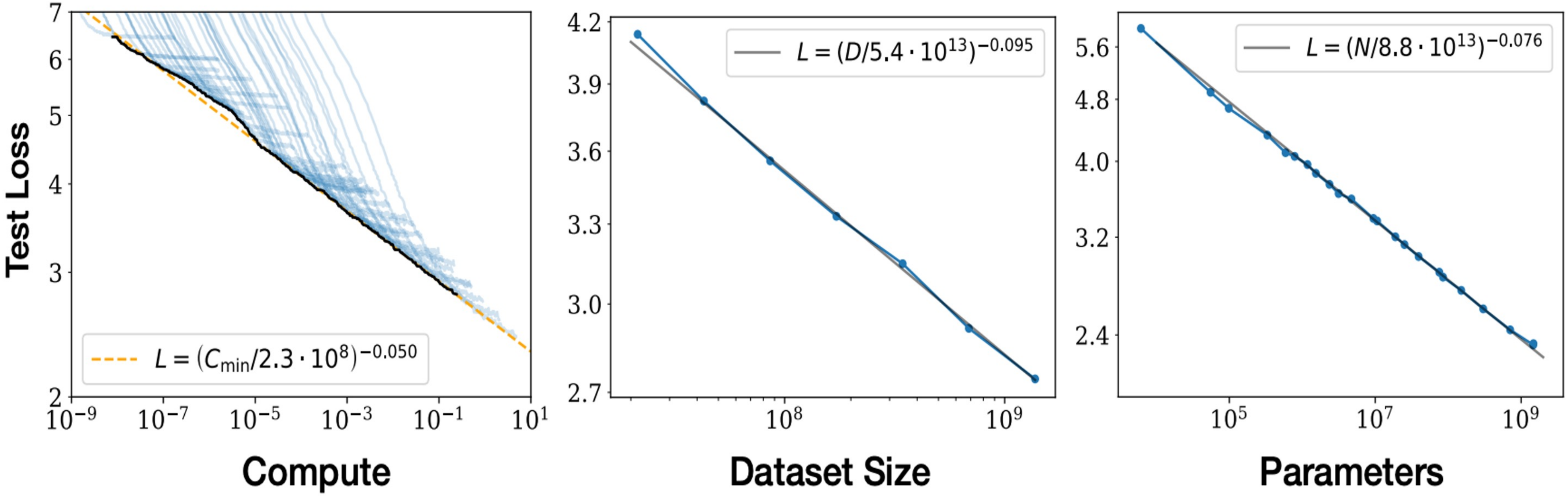
Training Compute-Optimal Large Language Models.
DeepMind. 2022.

Three key components of training LLMs (size >1B):

- **Model scale (N)**: number of parameters, layers and self-attention layers, and size of embedding size.
- **Dataset size (D)**: exhausting the Web data to pretrain LLM and curating a pretraining dataset takes time
- **Training Compute budget (C)**: most LMMs are undertrained.

LLM performance improves smoothly as we increase model size, dataset size, and the compute for training

Power-law relationship of performance ~ $(1/X)^{\alpha}$ for X=N,D,C

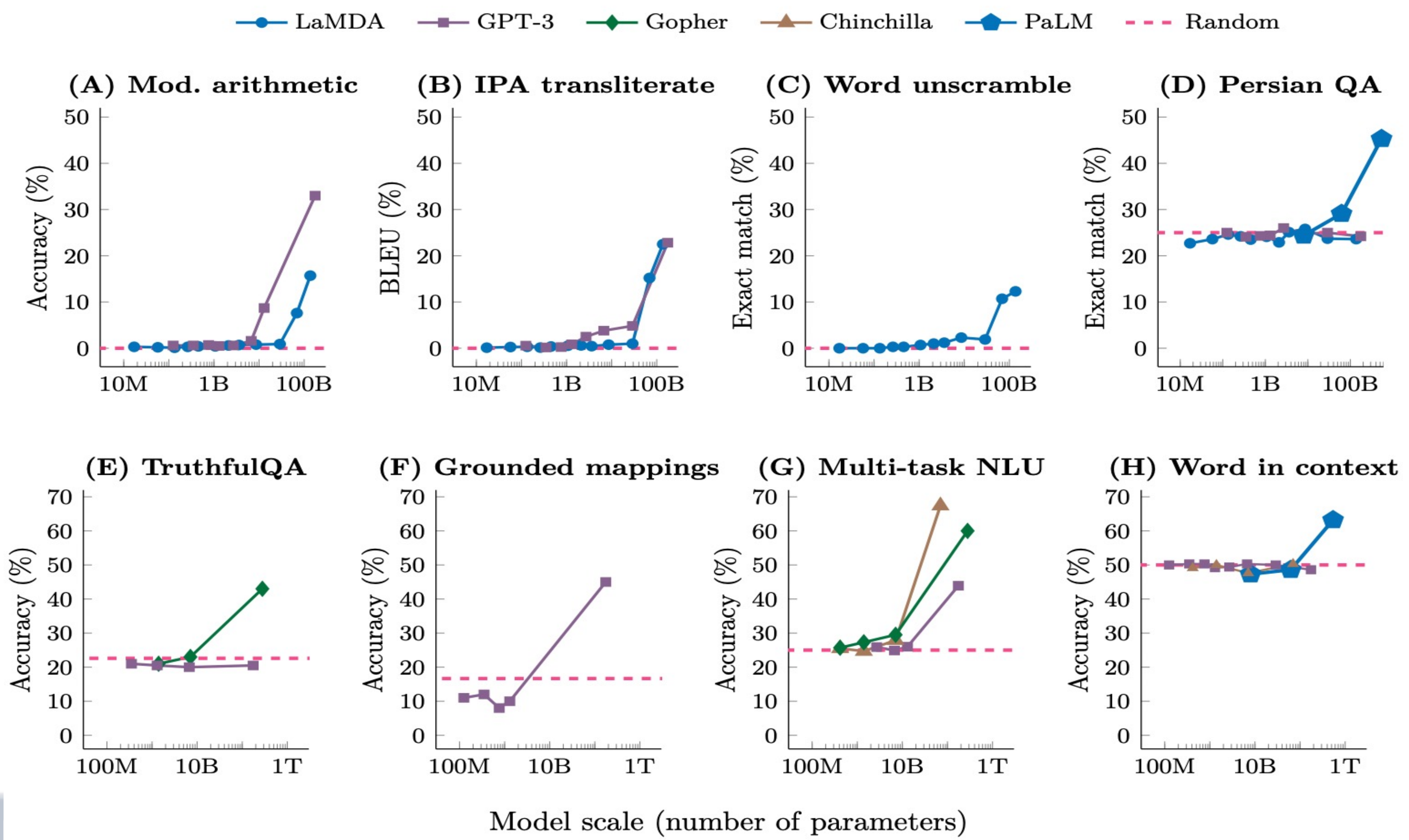


Emergent Abilities – the most prominent feature in LLMs

Emergent Abilities of Large Language Models. Google. 2022.

An ability to be ***emergent*** if it is not present in smaller models but is present in larger models.

The larger the better?



GPT-3 (2020)

Tom Brown, et. Language Models are Few-Shot Learners, https://doi.org/10.48550/arXiv.2005.14165, May 2020

- 175B parameters (1.5B - GPT-2), 96 layers, 96 attention heads, word embedding size 12888 (1600 - GPT-2), context size 2048 (1024 - GPT-2),
- few-shot learning without fine-tuning
- Data: ~500B tokens; 750+GB text from Webs, books, …
- Due to large model and extensive dataset, GPT-3 is a good text completion model, e.g., writing essays, text summarization, answering questions, language translation, and computer coding …

Generative AI system: A low variance (temperature closer to 0) may produce similar or repetitive samples, resulting in poor generation, whereas, a high variance (temperature closer to 1), may yield diversified but unrealistic or incoherent samples.

Let $z' = z/T$ where $T \in (0,1)$, prob of each word = softmax (z')

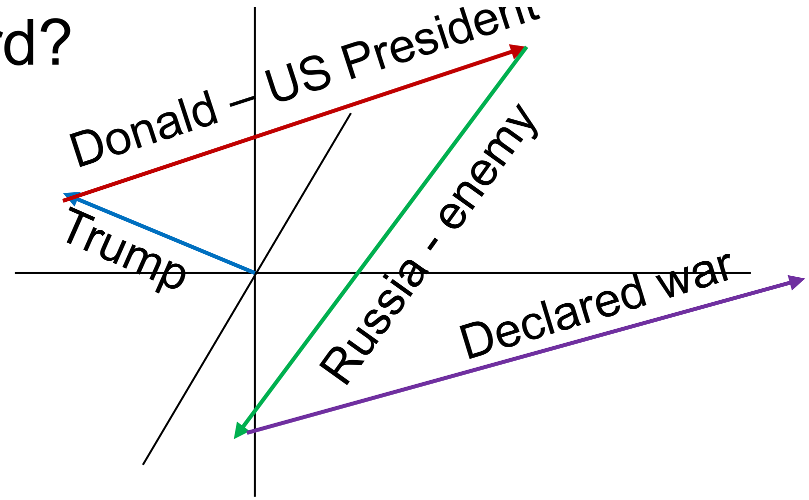
- Striking a balance between variation and fidelity is difficult, trade-off between exploring and exploiting the data distribution.

Next word prediction

- *Russia has declared war on the United States after Donald Trump accidentally* pressed the wrong button….

GPT3 : How to predict next word?

1. Trump – embedding
2. Attention with
   - Donald (US President)
   - Russia (competing country)
   - Declared war



Depending on attention and temperature

Next word	sat	pressed	fired	looked	spoke	and	a	the	…	
Probability %	2.5	2.35	2.3	2.01	1.5	0.05	0.09	0.08	…	

Context (GPT3 - 2048 words) more precise answer

Early version of ChatGPT forgets long conversation

Decoder – generate next word

Translation Source X → Target Y

$$Y_{best} = \text{argmax}_Y \, P(Y|X)$$

Exhaustive search is very expensive, B^s sentences

where B = number of branches, s = sentence length

Greedy Search: next word with the highest probability.

$$y_j = \text{argmax} \, P(y_j \,|X, y_1, \ldots, y_{j-1})$$

might miss the best sentence and prefer common words

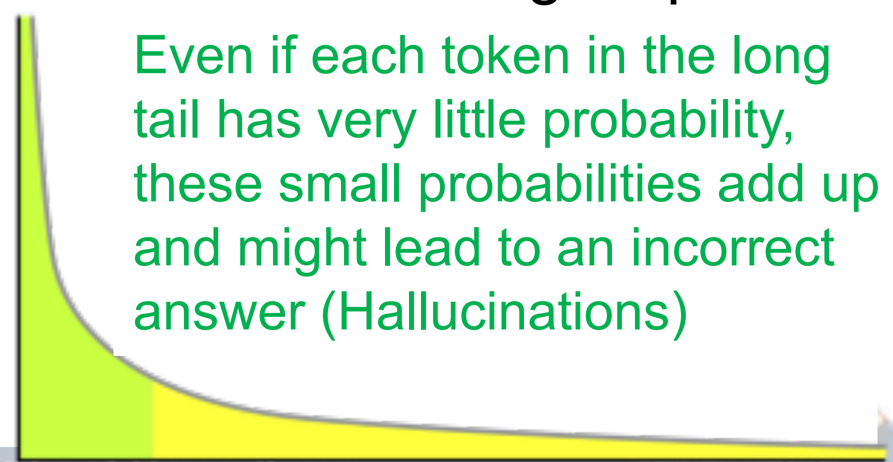
Beam Search: Breadth-first search on the top k (beam size) most probable sequences at each decoding step

Problem: Lack of Diversity

Sampling from Distribution

$$y_j \sim P(y_j \,|\, X, y_1, \ldots, y_{j-1})$$

- *Top-k, Top-p, ε- sampling, Stochastic Beam Search*



Even if each token in the long tail has very little probability, these small probabilities add up and might lead to an incorrect answer (Hallucinations)

Building LLM

- **LLMs are not intelligent, just statistical beasts.**
  - LLMs project a sense of “understanding” based on what they can do
  - They don’t *“understand”* text like humans do.
  - They predict the next most likely word based on vast amounts of training data. .
- **Building an LLM follows same 3-step logic as any ML model**
  1️⃣ **Collect massive amounts of data** (ideally the entire internet).
  2️⃣ **Train a baseline model** to capture patterns (autocomplete).
  3️⃣ **Improve the model** with fine-tuning to make it useful.
- **Baseline models are coherent but mostly useless.**
  Predicting next word makes them sound *real*, but they have no reasoning, truthfulness, or intent. It’s “just” pattern matching.
- **Fine-tuning turns a random word predictor into an actual tool,**
  i.e., knowledge, factual consistency, human alignment …

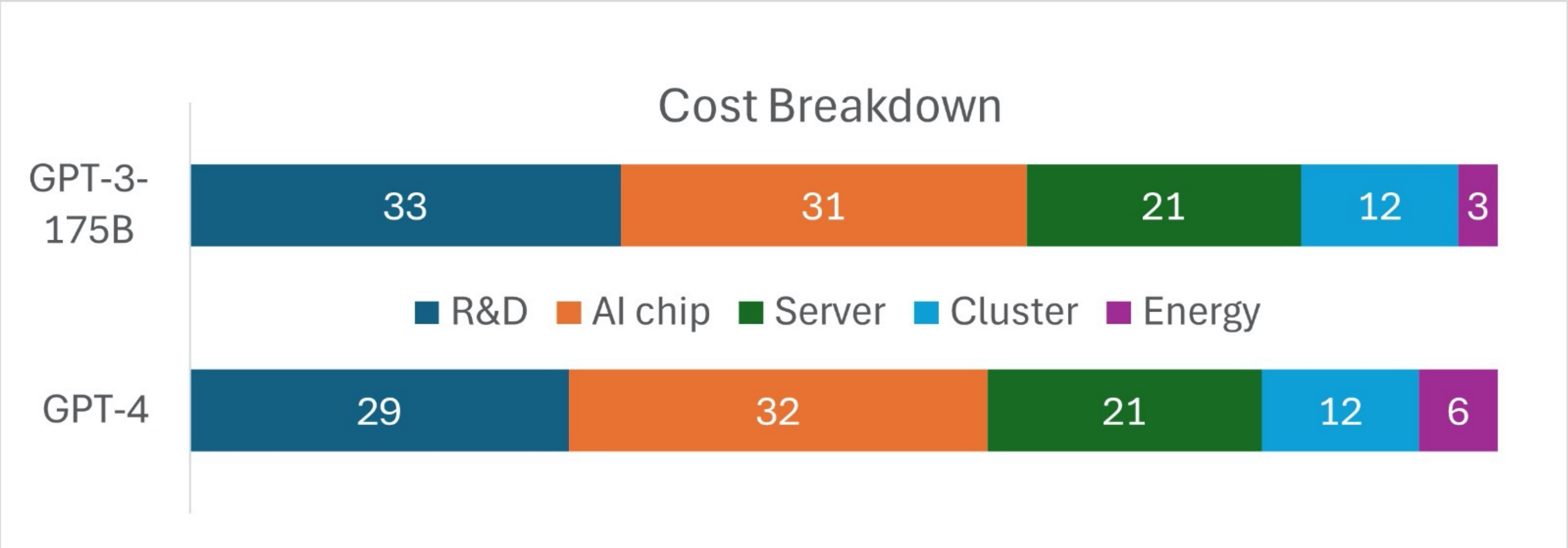
Baseline Model

A baseline LLM model is just a probability machine.
It learns to predict the next token based on statistical patterns in training data, not meaning or reasoning.

- It is trained to predict next word with loss $\mathcal{L} = -\log p_y$
- It writes coherent sentences but diverges and hallucinates.

Training costs are astronomical. Accessible to only a few major players, which is why NVIDIA stock skyrocketed.

- Training Cost for GPT-3 (175B) ~$4.6m, GPT-4 ~$80-100m



Improve the Model by Fine-tuning

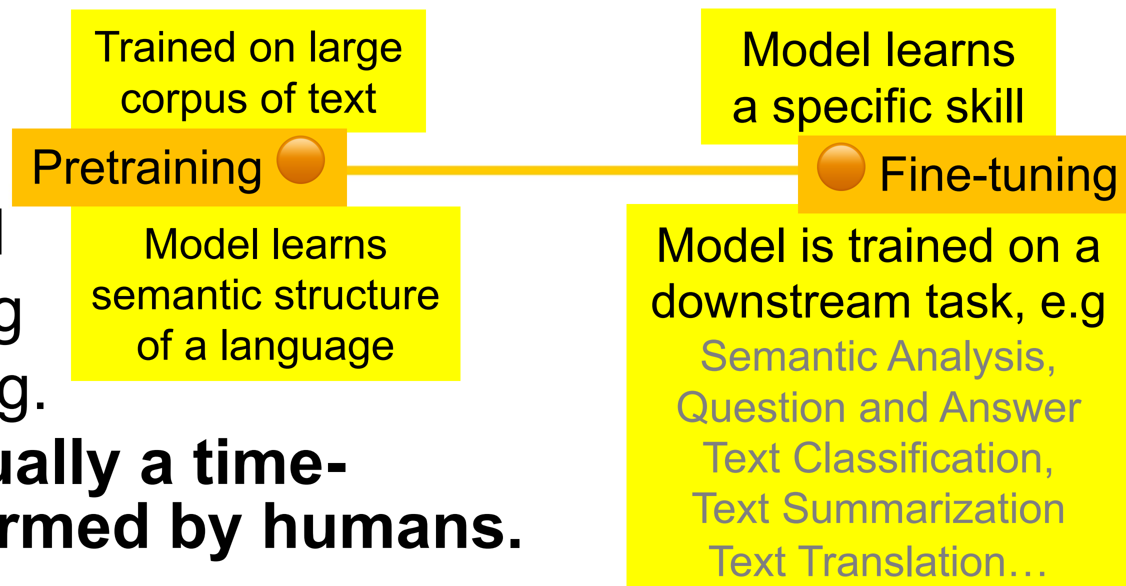
Aligning language models to follow instructions, https://openai.com/research/instruction-following, Jan 2022

GPT-3 is trained to predict next word on a large dataset, but cannot track user intent, i.e., not aligned with the users

- **Lack of helpfulness** - do not follow user's instructions.
- **Hallucinations** - reflect non-existing or incorrect facts.
- **Lack interpretability** - difficult for humans to understand how the model arrived at a particular decision/prediction
- **Toxic/biased content** - harmful/offensive misinformation

Fine-tuning requires a human-labeled dataset, large to allow the model to generalize its learning well and avoid overfitting.

Data annotation is usually a time-consuming task performed by humans.



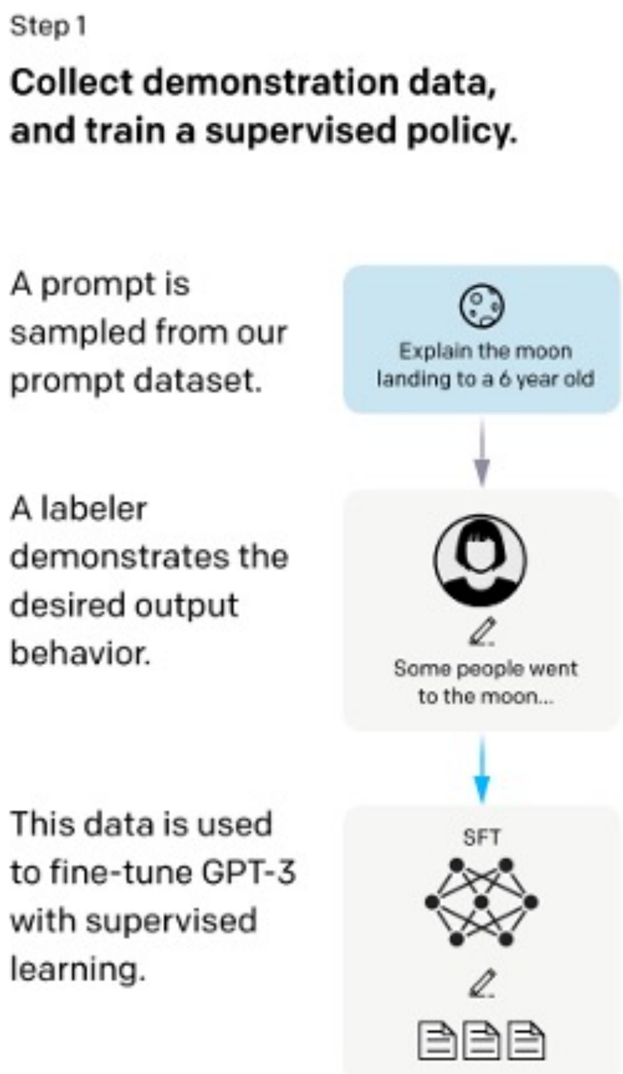
GPT-3.5 / InstructGPT (2022)

Aligning language models to follow instructions, https://openai.com/research/instruction-following, Jan 2022

Trained with humans to follow user intentions (human preference)

- RLHF: Reinforcement Learning from Human Feedback
  - Incorporate human expertise and knowledge
  - Mitigate untruthful, toxic output and hallucination
  - Make models more aligned with users
- Humans rank many different outputs produced by model for various prompts (human feedback in RLHF)

- **GPT-3.5 / InstructGPT**: a fined-tuned version of GPT-3
- **RLHF**: Reinforcement Learning from Human Feedback (Step 1: supervised fine-tuning (SFT))



Maximize diversity in the prompt dataset,
Only 200 prompts come from any given user
Any prompts that shared long common prefixes were removed.

Prompt dataset is a series of prompts previously submitted to the Open API

Three common prompts
Direct: "Tell me about…"
Few-shot: Given these two examples write another story on the same topic.
Continuation: Given the start of a story, finish it.

40 contractors hired to write responses to prompts

Example:
"What is the UK's capital?"
The annotator needs to give "the UK's capital is London"

Input / output pairs are used to train a supervised model on appropriate responses to instructions.

GPT-3.5 / InstructGPT (2022)

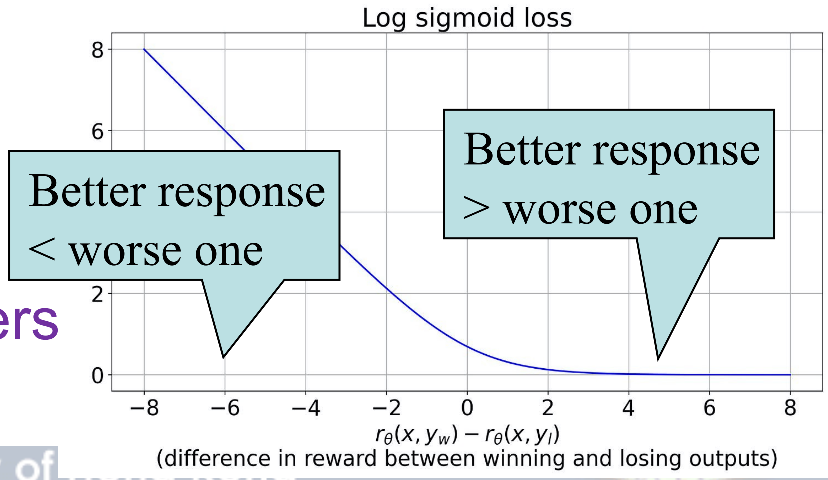
Aligning language models to follow instructions, https://openai.com/research/instruction-following, Jan 2022

Trained with humans to follow user intentions (human preference)

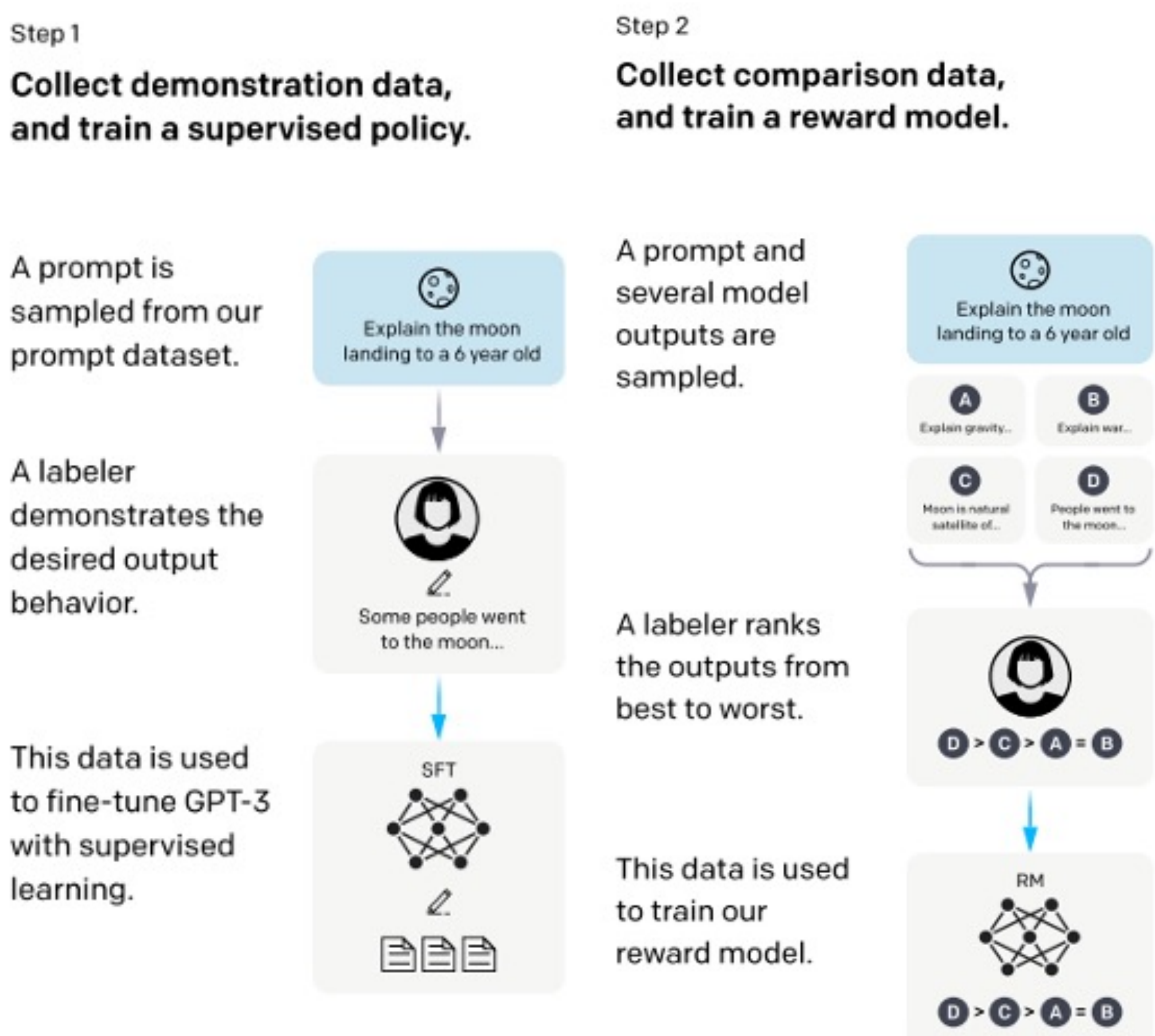
- RLHF: Reinforcement Learning from Human Feedback
  - Incorporate human expertise and knowledge
  - Mitigate untruthful, toxic output and hallucination
  - Make models more aligned with users
- Humans rank many different outputs produced by model for various prompts (human feedback in RLHF)
- Train a reward model that predicts which output humans will prefer
- Choosing best answer among two options is a much simpler task than providing an exact answer to an open question.
- Reward model generates positive values for good responses and negative values for bad responses.

$\mathcal{L}(\theta) = -\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l))$

prompt x, outputs y_w that human prefers and y_l that they don't like as much.



- **GPT-3.5 / InstructGPT**: a fined-tuned version of the GPT-3
- RLHF: Reinforcement Learning from Human Feedback (Step 2: reward model (RM) training)



Collecting human data is expensive and slow.

Responses are generated by the SFT model

Step 2 trains a reward model to replace human judgments.

Step 3 uses the model to interact with reward model (a simulated human annotator).

$\binom{k}{2}$ combinations of rankings served to the model as a batch datapoint

ChatGPT (2022) https://chat.openai.com/

- Large Language Model (LLM) based on GPT3 –
  - 175 billion parameters, 96 layers, 96 attention heads, embedding size 12888, input size 2048
  - Pretrained with data from web (~ 500 billion tokens; 750+GB of text)
- ChatGPT:
  - embedding size 1536 vs 12888 (GPT-3)
  - parameters 20 billion vs 175 billion (GPT-3)
  - input size 8192 vs 2048 (GPT-3)

Embedding of input → Decoder → Output

Learn to understand meaning and relationship of input words (very precise description through large embedding size and many attention heads)

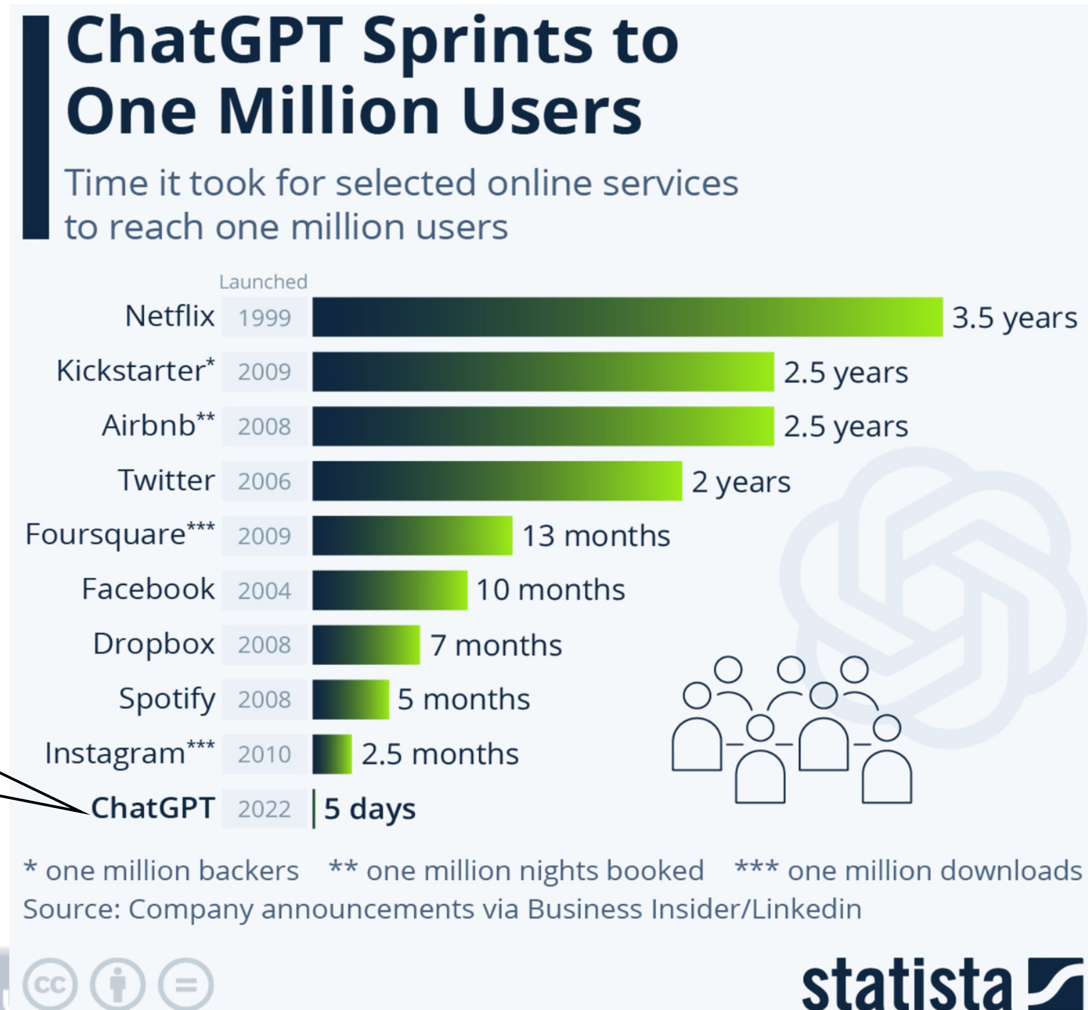
Generate fluent and coherent text from transformer decoder (depends on large model, lots of training data and good input embedding)

is a Big Hit

November 2022

- ChatGPT gained 1 million users in only 5 days
- Over 100 million monthly active users within 2 months of launch

--- Tik Tok reached 100 million in 9 months



Fastest rising Application!

ChatGPT (2022) https://chat.openai.com/

- Predict the next word is not enough to make chatbot, e.g.,
  - if you asked GPT-3 to "Write an article about American football"
  - predicting next word
    "…. and its influence on television in America."
- Conversation Protocol **finetuning** – specialized on prompting
  - User: Hi! I am a human
  - Assistant: Hello! Nice to meet you! I am ChatGPT, your AI assistant.

  create new tokens that represent an action,
  e.g., OpenAI uses system tokens like <|im_start|> & <|im_end|>

Based on InstructGPT, ChatGPT incorporates (1) human feedback during training using reinforcement learning (2) slight differences in data collection setup and trained **specially optimized for dialogue**.

- Step 1: Train an InstructGPT model using Reinforcement Learning from Human Feedback (RLHF)
- Step 2: Human trainers use model-written suggestions to compose dialogue responses.
- Step 3: Mix this new dialogue dataset with the InstructGPT dataset in a dialogue format.

ChatGPT – Helpfulness vs. Harmlessness

ChatGPT is a strong, successful and popular AI tool

- helpful for many real-world NL tasks like Q&A, writing emails, making excel forms, writing programs, polishing essays, summarising complex research papers, analysing legal documents, writing poetry in different styles,…
- can generate human-like responses on science, history, or literature (surprising accuracy or totally made-up stuff).
- there is a tradeoff between helpfulness and harmlessness.
  - stress more on helpfulness, tend to increase harmfulness.
  - stress more on harmlessness, tends to refuse to answer in many cases. This reduces helpfulness.
- need **balance between helpfulness and harmlessness.**

ChatGPT - Limitations

1. plausible-sounding but **incorrect/nonsensical answers**
   – RL training gives no source of truth (Hallucination);
   – training the model more cautious might decline correct answers (Helpfulness vs. Harmlessness)
   – supervised training based on available data, not what human knows
2. **sensitive to tweaks** to input phrasing or attempting the same prompt multiple times.
3. **excessively verbose** and overuses certain phrases, from biases in the training data (trainers prefer longer answers that look more comprehensive)
4. guess user's intent (instead of asking for clarifications) for **ambiguous query**.
5. respond to **harmful instructions** or exhibit **biased behavior**.

ChatGPT – Importance of Data Privacy

- Jasper AI (https://www.jasper.ai/) provides AI copywriter services without their own in-house models.
- Jasper is an early customer of OpenAI APIs.
- OpenAI found that Jasper uses its APIs to write emails and essays.
- ChatGPT enhances its abilities in writing emails and essays.
- People turn to use ChatGPT instead of Jasper.
- This raises issues about data privacy when using ChatGPT or large language models.

Note: OpenAI will record every prompt you sent to the OpenAI APIs!

GPT4 (2023)

GPT-4 visual input example, Extreme Ironing:	
User	What is unusual about this image?

Source: `https://www.barnorama.com/wp-content/uploads/2016/12/03-Confusing-Pictures.jpg`

GPT-4	The unusual thing about this image is that a man is ironing clothes on an ironing board attached to the roof of a moving taxi.

Table 16. Example prompt demonstrating GPT-4's visual input capability. The prompt requires image understanding.

GPT-4 (2023)

GPT-4 Technical Report, March 2023, https://arxiv.org/pdf/2303.08774.pdf

- GPT-4 is a large multimodal model (image and text)
- No technical details were released (Section 2 in the paper):

  "This report focuses on the capabilities, limitations, and safety properties.…Given both the competitive landscape and the safety implications of large-scale models like GPT-4, this report contains no further details about the architecture (including model size), hardware, training compute, dataset construction, training method, or similar. …" (guess:1.7T, 120att)

  86B neurons with 100T parameters (human brain)
- Passes a simulated bar exam with a score around top 10% of test takers; GPT-3.5's score was around bottom 10%.
- GPT-4 can generate responses of more than 25,000 words, while GPT-3.5 is limited to about 3,000-word responses.
- 82% less likely to respond to requests for disallowed content 40% more likely to produce factual responses than GPT-3.5
- GPT-4 with context windows of 8192 and access 32768 tokens, while GPT-3, which were limited to 4096 and 2048 tokens

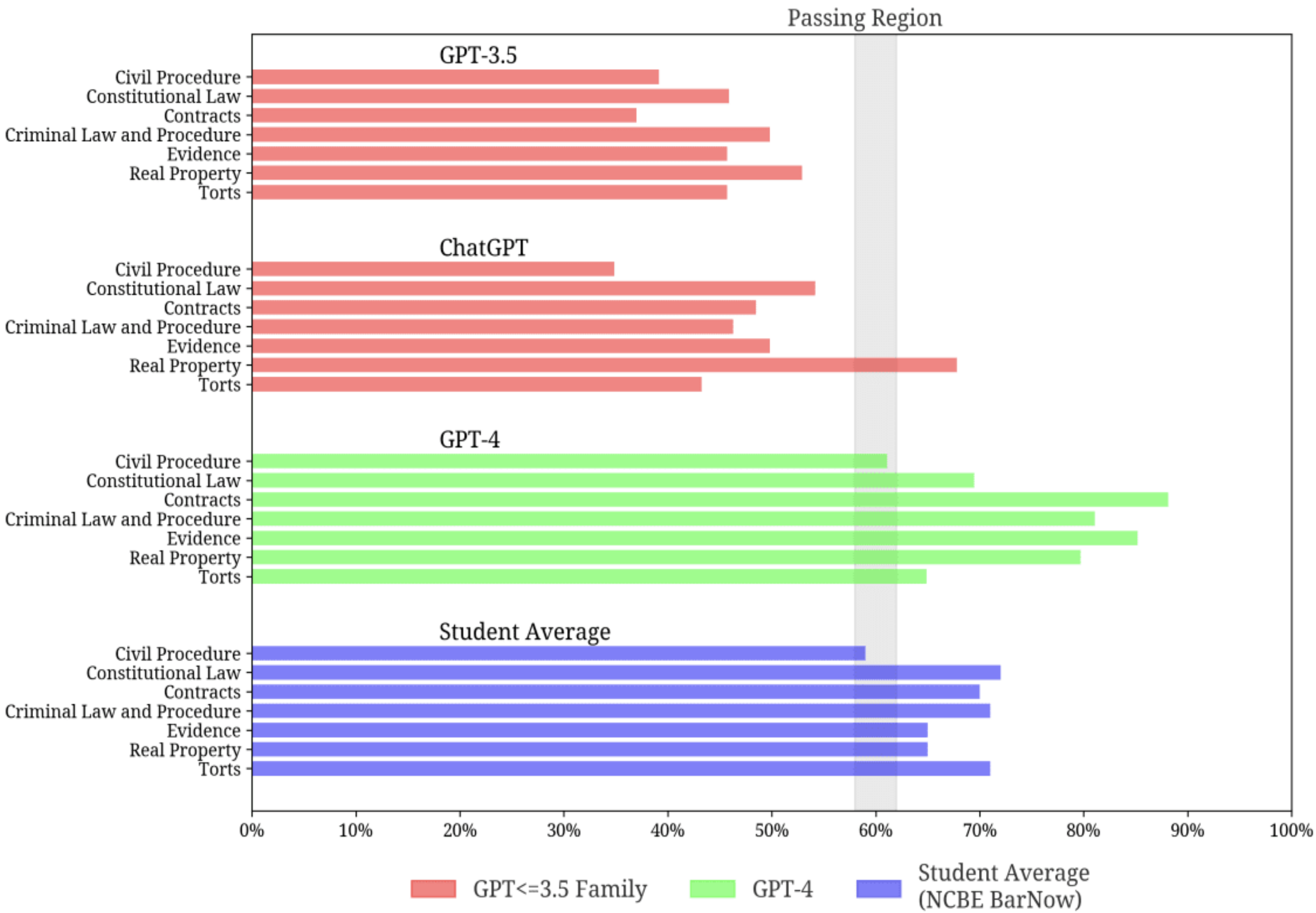
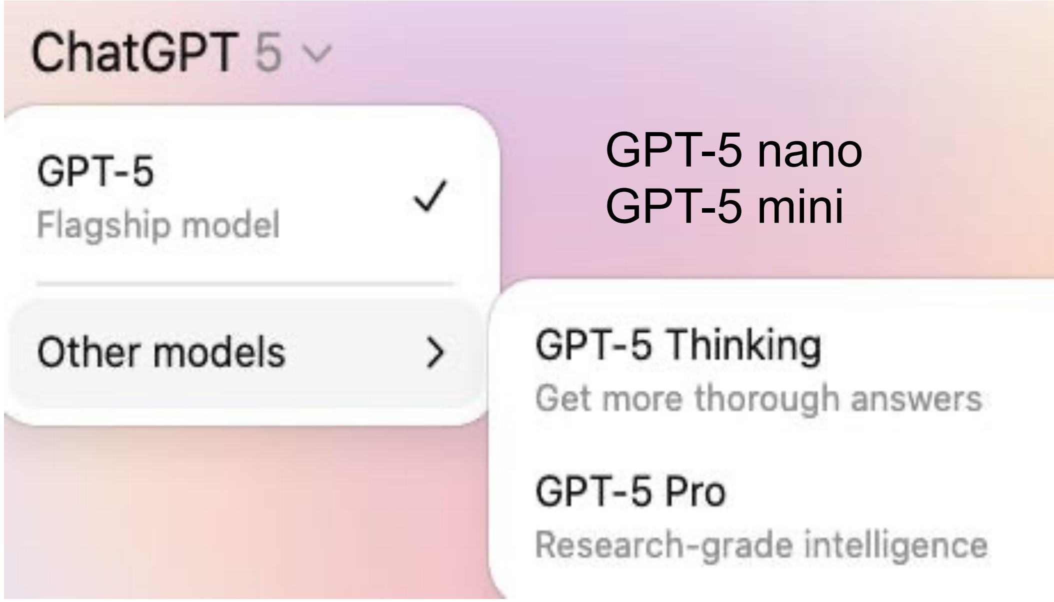


Figure 2. Progression of Recent GPT Models by Legal Subject Area

GPT-5 (2025)

Three Upgrades

1. Model selector - no switching between different models, unified system decides how much processing power to apply based on the complexity of the request.



2. Fewer hallucinations - 80% fewer factually incorrect statements; simply answer such as “I can’t do that,”
3. Create apps in seconds (even without a programming background) and perform well in tests, math competition

Other Upgrades - beyond grammar, correctness, tone, style,

- Medical advice - raise concerns, ask clarifying questions, and adjust its explanations based on the user’s context.
- Multimodal – interpret chart, diagram, photo, presentation

How LLMs collect and clean training data

- **Garbage in, garbage out** applies to LLMs.
  vast, diverse, and well-structured training data to be effective.
- **LLMs need trillions of tokens** for models of trillion parameters

GPT4	1.8T parameters	13T tokens (9.7T words)
Deepseek V3	0.6T parameters	15T tokens (11.1 words)

- **Not all internet text is valuable**. (45T -> 570G) Webpage formatting, spam, duplicated content, misleading information

Dataset	Size (GB)	Training Weight	epoch
Wikipedia	3	3%	3.40
WebText	20	21%	2.90
books	67	16%	2.33
crawl	400	60%	0.44

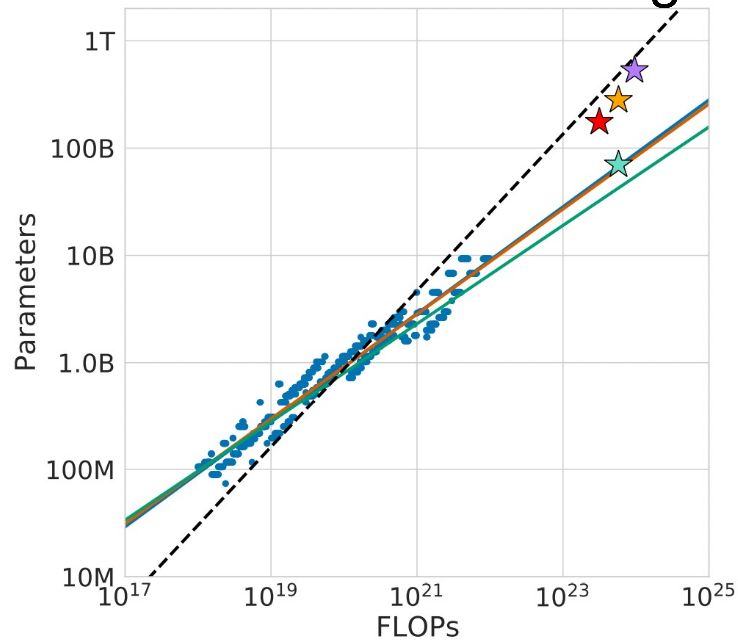
- Common crawl – messy, noisy, duplicate, but open, largest free data, 250B pages (3-5B new pages / month) – Fineweb project
- **Pre-training isn't enough**. After collecting and cleaning data, companies refine it with careful filtering and deduplication before even starting model training.

Optimal Large Language Models

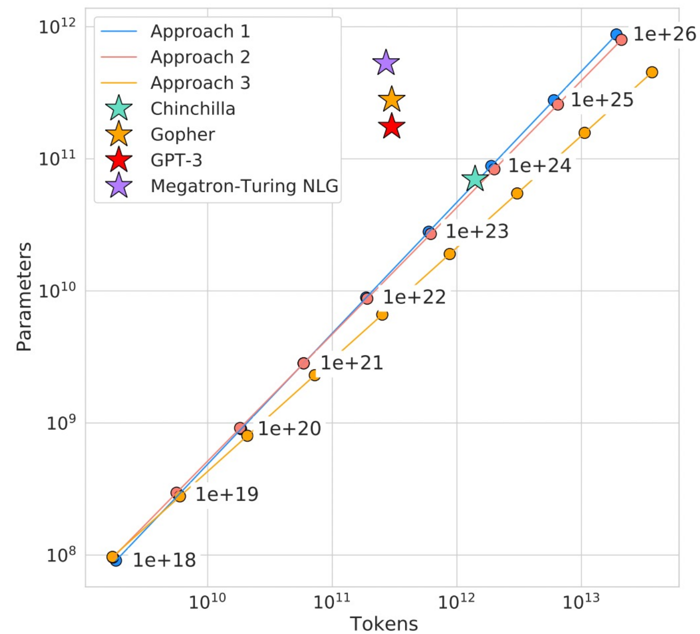
- Large models are more **sample-efficient** than small models, i.e., reaching same performance with fewer training steps.

How many data do we need to train a large model?

- Model size and training data should be scaled equally i.e., doubling of model size, doubling of training data
- A large model is not necessary if not enough training data.
- Current large models should be substantially smaller and therefore trained much longer than is currently done



Optimal Compute Given the Model Scale



Optimal Data Size Given the Model Scale